

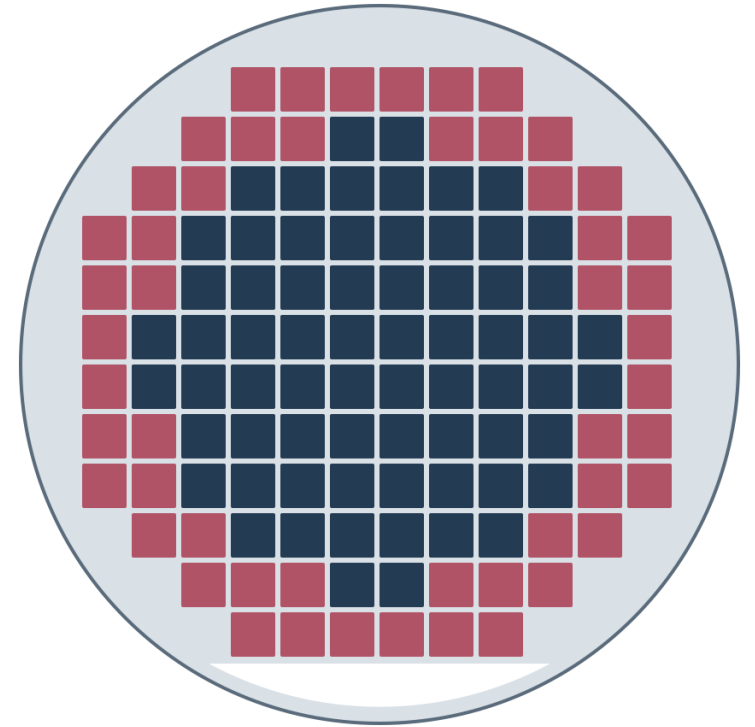
CS2.501

Advanced Computer Architecture

Lecture 1 · Moore's Law and the End of the Free Lunch

Suresh Purini · Priyesh Shukla

Monsoon 2026



300 mm wafer, schematic — red dies mark edge yield loss

What we will do in the next 90 minutes

Part 0	Course mechanics	Outcomes, syllabus, grading, tools, expectations
Part 1	What is computer architecture?	ISA vs. microarchitecture; the abstraction stack
Part 2	Moore's Law	What Moore actually said, and what the data says now
Part 3	Dennard Scaling	Why clocks rose, and why they stopped
Part 4	The three walls	Power, ILP, Memory — and Amdahl's ceiling
Part 5	The new golden age	Domain-specific architectures and what comes next



Course Mechanics

Outcomes, syllabus, assessment, and what is expected of you

Course at a glance

Course

CS2.501 — Advanced Computer Architecture

Instructors

Suresh Purini
Priyesh Shukla

Office hours by appointment.

Textbook

Hennessy & Patterson,
Computer Architecture:
A Quantitative Approach, 6th ed.

Plus conference papers.

Prerequisites

- Computer Programming — you will write and instrument real code.
- Computer Systems Organization or Introduction to Processor Architecture — pipelines, caches, assembly.
- Comfort with basic probability and with reading a graph on a log scale.

Takeaway If you have never drawn a 5-stage pipeline or written a cache simulator, talk to us this week.

Course outcomes

What you should be able to do in December that you cannot do today

CO-1

Explain the principles and practices underlying the design of modern processors.

Cognitive level
Understand

CO-2

Explain the principles and practices underlying the design of memory hierarchies.

Cognitive level
Understand

CO-3

Analyse a processor/memory architecture qualitatively and quantitatively, and develop strategies to mitigate observed bottlenecks.

Cognitive level
Apply · Analyze · Evaluate · Create

CO-4

Explain and identify security vulnerabilities in modern computing systems, and propose techniques to circumvent them.

Cognitive level
Understand

Six units, one argument

Each unit is a response to a limit we will meet today

Unit 1

Instruction Level Parallelism & Superscalar Processors

Out-of-order microarchitecture;
compiler/architecture interaction

Unit 2

Data Level Parallelism & SIMD / GPU Architectures

Vector architectures; GPUs; compiler
interaction

Unit 3

Memory Hierarchy & Emerging Memory

Caches; distributed shared memory;
consistency; NVM

Unit 4

Emerging & Domain-Specific Accelerators

DNN and genomics accelerators; AMD AI
Engine; TPUs; near-memory

Unit 5

AI/ML Techniques in Architecture

RL for cache replacement and branch
prediction; power management

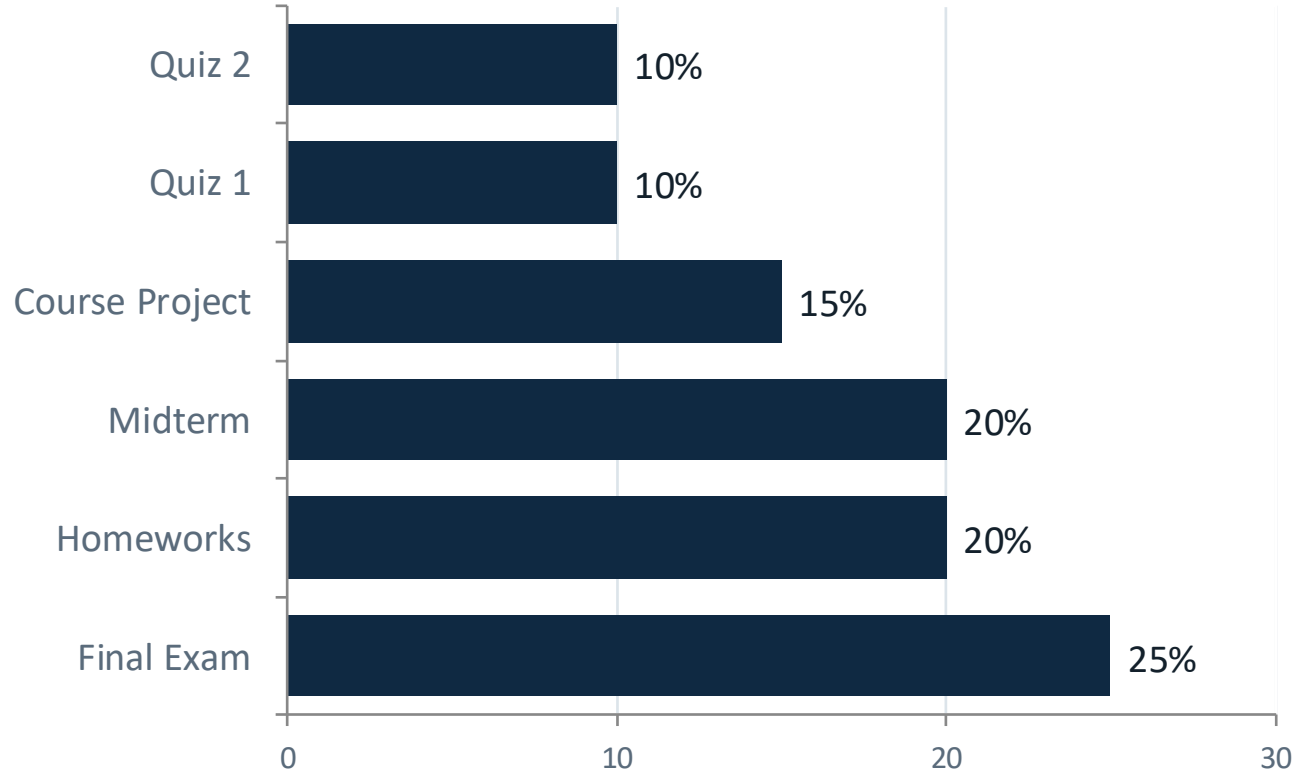
Unit 6

Security

Side-channel attacks: Spectre, Meltdown, and
their descendants

Assessment

Total 100% — continuous evaluation, no single make-or-break exam



Homeworks (20%)

Analytical problems plus simulator work. Expect to profile real binaries with PIN and to drive cache simulators.

Course Project (15%)

Team project. Reproduce, extend or critique a result from ISCA / MICRO / HPCA. Proposal due mid-semester.

Takeaway 60% of your grade is decided before the final exam. Start strong.

How this course is taught

This is a research-preparation course, not a survey course

Read the primary literature

The textbook gives you vocabulary. The field lives in ISCA, MICRO, HPCA, ASPLOS. By the end of the semester you should be able to read one of these papers and say what is actually new in it.

Measure, do not guess

Every claim about performance is an empirical claim. We will use PIN for dynamic binary instrumentation and memory-hierarchy simulators to test intuitions against data.

Quantitative, always

"X is faster" is not an answer. Speedup relative to what baseline, on what workload, at what power, at what cost?

Design under constraint

Every architecture idea we study is a trade against area, power, latency, bandwidth and verification cost. Learn to name the trade.

1

What Is Computer Architecture?

The art of choosing what to build with the transistors you are given



Three things people mean by "architecture"

Instruction Set Architecture

The contract with software: registers, addressing modes, instruction semantics, memory model, exceptions.

x86-64, ARMv9, RISC-V

Microarchitecture

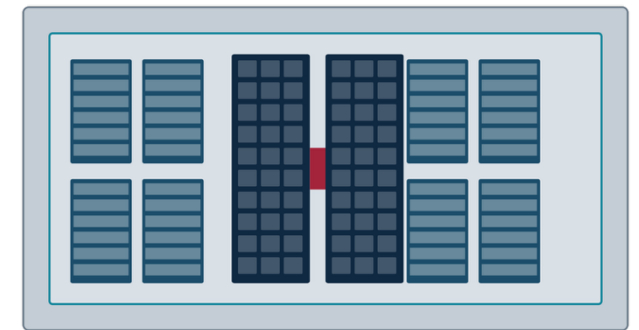
How that contract is implemented: pipeline depth, issue width, ROB size, branch predictor, cache organisation.

Zen 5, Golden Cove, Apple Firestorm

Implementation / Realisation

Logic design, circuits, process node, packaging, thermal envelope.

TSMC N3, chiplets, CoWoS



Multi-die package: two compute dies, eight memory stacks

Takeaway One ISA, many microarchitectures. This course is overwhelmingly about the second and third rows.

The stack, and where the leaks are

Abstraction layers are how we ship systems — and where performance and security go to die

Application
Algorithm
Programming language / Compiler
Operating system / Runtime
Instruction Set Architecture
Microarchitecture
Logic / Circuits
Devices / Process technology

Two claims for the semester

- Performance is a whole-stack property. A 10x algorithmic win beats a 1.2x microarchitectural win — but you need both, and they interact.
- Abstractions leak. Speculation is invisible in the ISA and catastrophically visible in the cache. That leak is Spectre (Unit 6).
- Compilers and architecture co-evolve. Units 1 and 2 both have an explicit compiler-interaction topic for this reason.

Why take this course in 2026?

For fifty years, software got faster because hardware got faster, and hardware got faster for reasons that had nothing to do with software.

That arrangement has ended. Performance is now an architectural problem — which means it is your problem.

~3%

annual single-thread
performance gain today

~52%

annual gain during
the RISC/ILP era

10^5

growth in training compute
for frontier AI models
over the last decade

336B

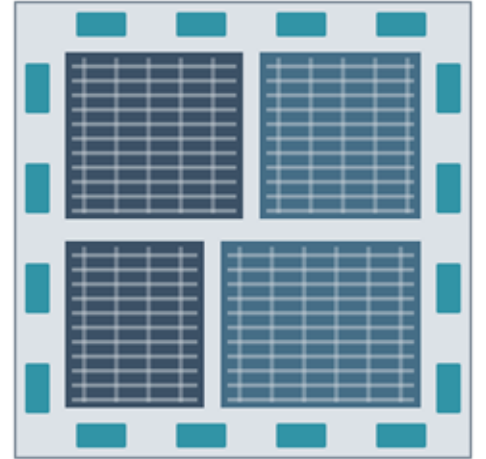
transistors in a single
2026 AI accelerator

Takeaway Demand for compute is growing exponentially at exactly the moment supply stopped being free. Architecture is the gap.

2

Moore's Law

An observation about economics that became a fifty-year industry roadmap



Gordon Moore, Electronics magazine, April 1965

Four data points and a bold extrapolation

What he actually observed

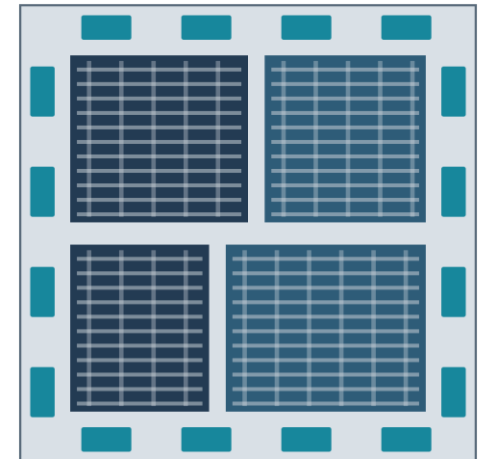
Plotting the number of components per integrated circuit that gave the minimum cost per component, against time, Moore found the count had roughly doubled each year from 1959 to 1965 — and predicted this would continue for at least a decade.

The 1975 revision

At IEDM 1975, Moore revised the rate to a doubling roughly every two years, distinguishing gains from die area, feature-size shrink and "device cleverness". The two-year figure is the one that held.

Three things Moore's Law is not

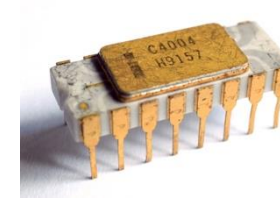
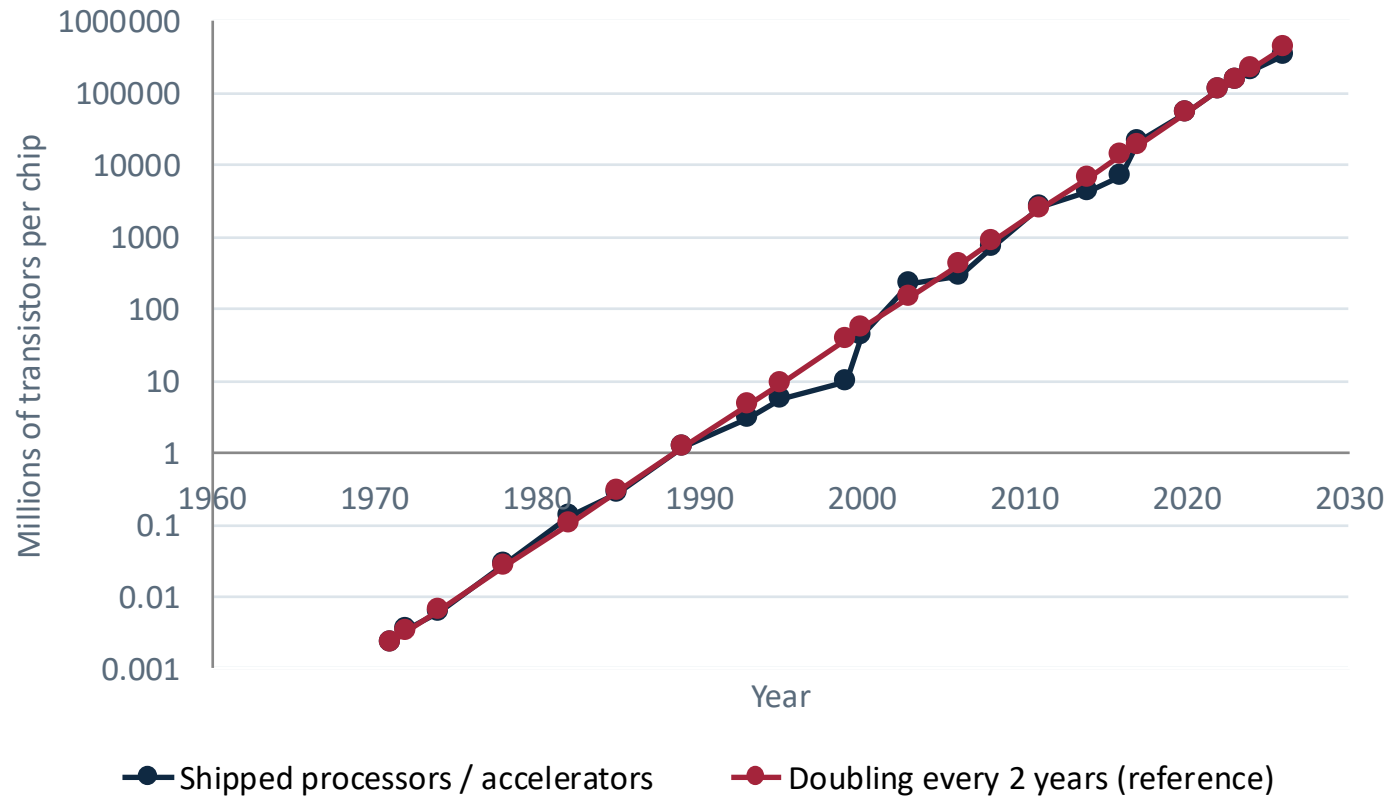
- It is not a law of physics. It is an observation about an industry that then organised itself to make the observation true.
- It is not about speed. Moore said nothing about clock frequency or performance — only about component count at minimum cost.
- It is not about performance per dollar of a finished system. It is about cost per transistor at the optimal die size.



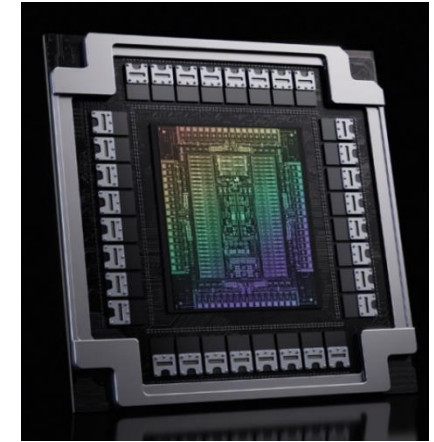
1971-era die: 2,300 transistors

Fifty-five years of transistor counts

Log scale. A straight line here means constant exponential growth.



1971 · 2,300



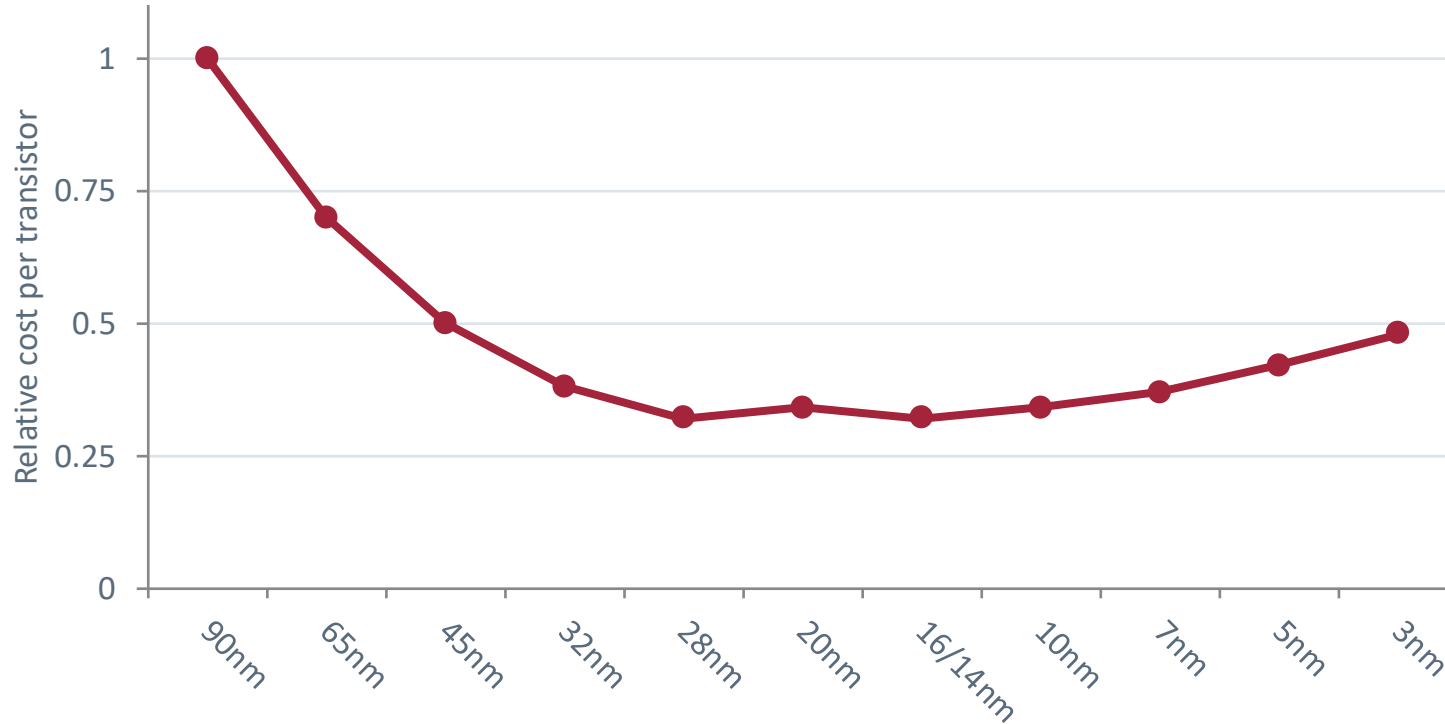
2026 · 336 billion

- 4004 (1971): 2,300 transistors.
- NVIDIA Rubin (2026): ~336 billion.
- That is a factor of $\sim 1.5 \times 10^8$ in 55 years.
- **The trend survived — but note what is on the recent end of the curve: GPUs and multi-die packages, not single CPU cores.**
- Growth after ~2016 increasingly comes from chiplets, 2.5D/3D integration and reticle stitching, not from density alone.

Representative parts; counts as reported by vendors. Multi-die packages counted as one chip.

The part of Moore's Law that broke first

Moore's claim was about cost. Cost per transistor stopped falling around 28 nm.



- **Why it turned**

- Multi-patterning, then EUV: more masks, more steps per layer.
- Design and verification cost per node now runs into hundreds of millions of dollars.
- Yield on very large dies falls with area.

- **Contested**

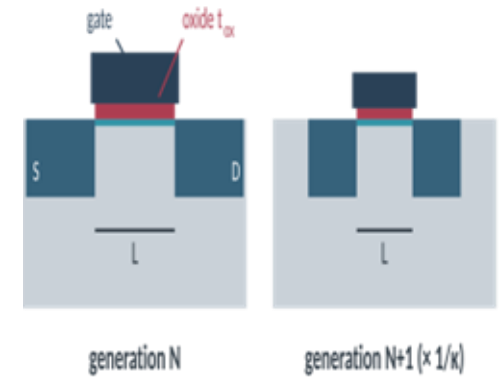
- Foundries dispute the exact shape of this curve. Treat the numbers as indicative, the qualitative flattening as well-established.
- This is why chiplets exist: build the big thing out of small, high-yield dies.

Takeaway Transistors keep shrinking. They have stopped getting cheaper. Moore's economic law bent before his density law did.

3

Dennard Scaling

The reason smaller transistors used to mean faster, cooler chips — and why that stopped



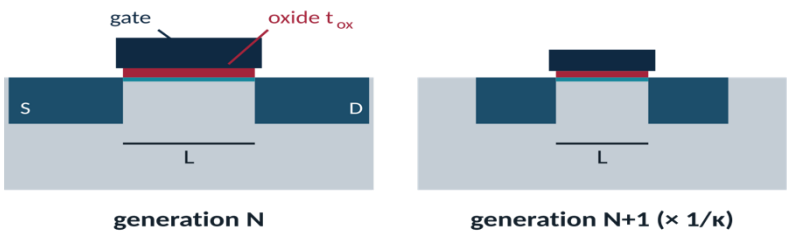
Dennard scaling, 1974

Scale every dimension and the voltage by the same factor $\kappa > 1$

Quantity	Scales by	Effect
Length, width, oxide thickness	$1/\kappa$	Smaller
Supply voltage V^{DD} , threshold V_t	$1/\kappa$	Lower
Doping concentration	κ	Higher
Capacitance $C \propto \text{area}/\text{thickness}$	$1/\kappa$	Lower
Delay $\propto CV/I$	$1/\kappa$	Faster
Frequency f	κ	Higher
Power per transistor $\approx CV^2f$	$1/\kappa^2$	Lower
Transistors per unit area	κ^2	Denser
Power density	1	Unchanged

The bargain

- Each generation: $\sim 2\times$ the transistors, $\sim 40\%$ faster switching, and the same total power in the same area.
- Software did nothing and got faster every 18 months.
- This — not Moore's Law — is what people actually meant by "free lunch".**
- Held remarkably well from roughly 1974 to roughly 2004.



Every dimension and the voltage shrink by the same factor

Why Dennard scaling ended

Voltage stopped scaling, and leakage refused to be ignored

$$P = \alpha C V^2 f + V \cdot I_{\text{leak}}$$

dynamic (switching)

static (leakage)

1. V_t cannot fall

Sub-threshold leakage grows exponentially as the threshold voltage drops. Lower V_t means a transistor that never truly turns off, so V_t stalled — and V^{DD} had to stall with it to preserve noise margin and drive strength.

2. V^2 was the whole win

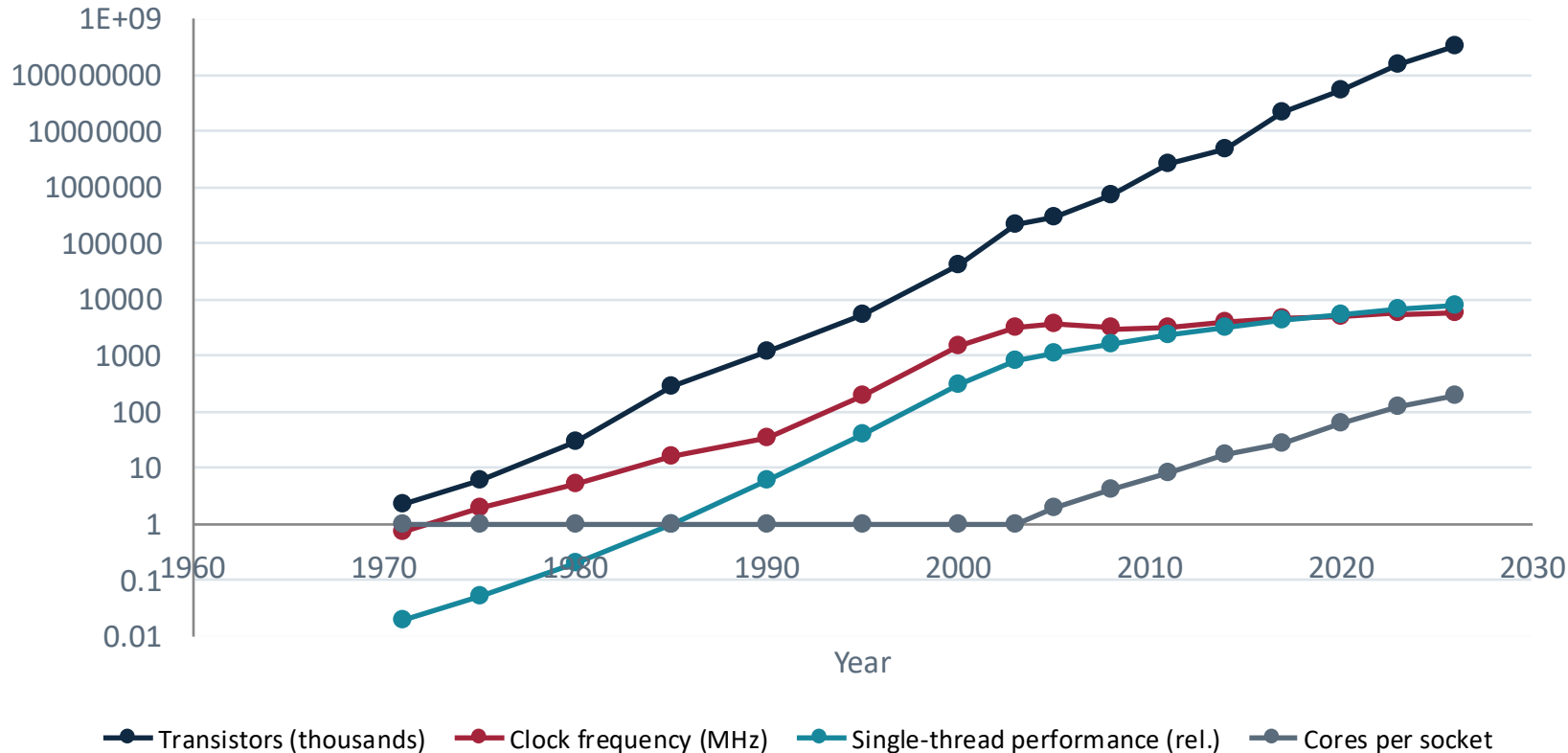
Dynamic power falls with V^2 . Freeze V and the $1/\kappa^2$ power reduction per transistor vanishes. Density still rises as κ^2 , so power density now rises as κ^2 instead of staying flat.

3. Leakage is first-class

Thin gate oxides leak by tunnelling; short channels leak sub-threshold. Static power became a significant fraction of the budget, and it is paid whether or not the transistor does useful work.

Fifty years of microprocessor trends

Four quantities that used to rise together, and stopped in about 2005

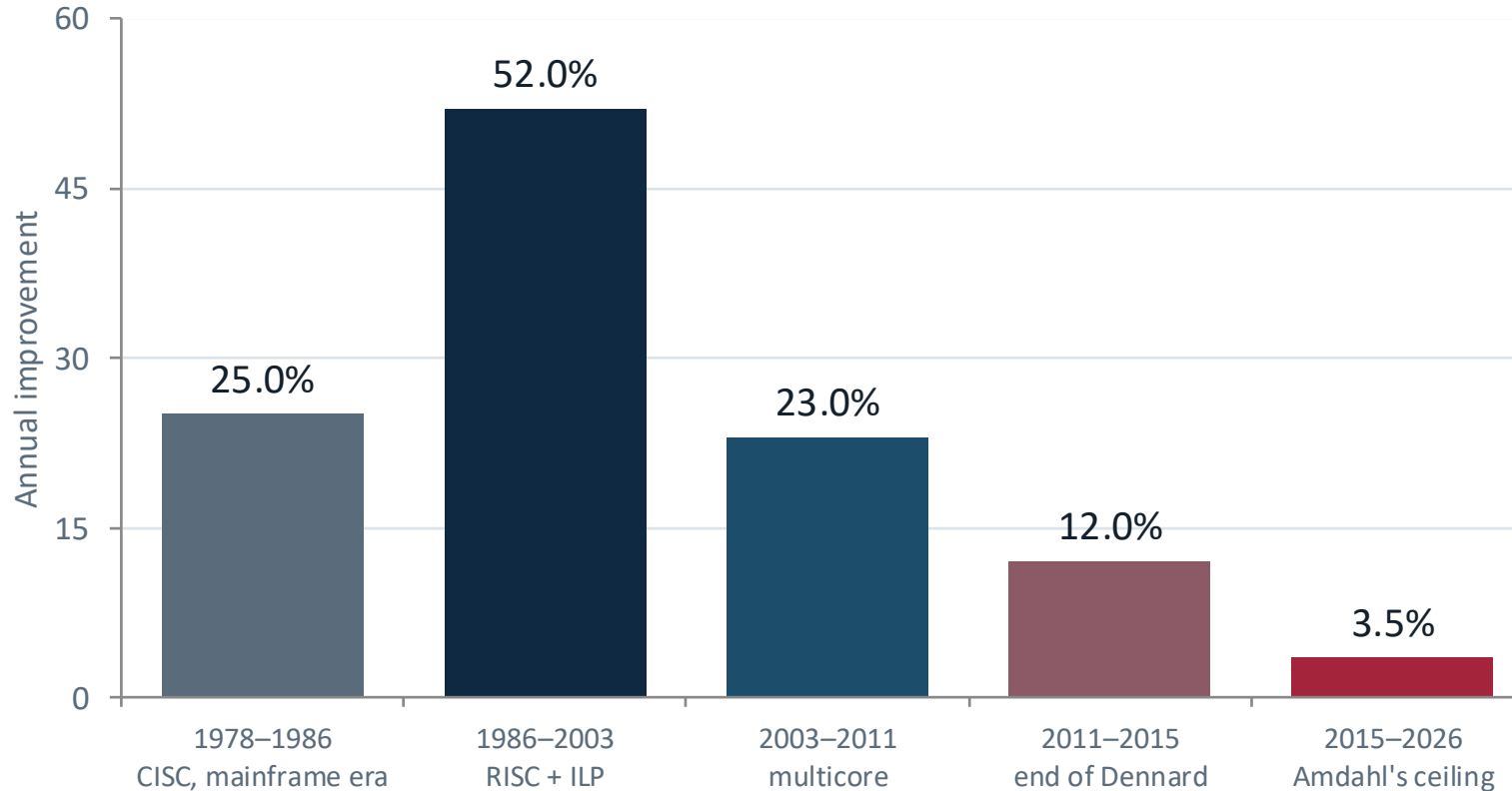


- **Read this chart in two halves**
- Before ~2005: all four curves climb together. Transistors buy frequency, frequency buys single-thread performance, core count stays at one.
- After ~2005: frequency flattens at 3–6 GHz. Single-thread performance bends. Core count takes off.
- Transistors keep coming. What changed is what we are allowed to do with them.
- That inflection is the reason Units 2, 3 and 4 of this course exist.

Indicative data compiled from vendor specifications and published SPEC results; single-thread performance normalised.

Growth in single-processor performance

Compound annual growth rate by era



- **What 52% per year meant**
- A machine bought in 1990 was obsolete in three years. Waiting was a valid optimisation strategy.
- **What 3.5% per year means**
- Doubling time is now about 20 years. Waiting is no longer a strategy.
- If your workload needs to be 100× faster, the hardware will not deliver it for you. Someone has to design for it.
- **That someone is an architect.**

4

The Three Walls

Power, ILP, Memory — and the ceiling Amdahl put above all of them



Three walls, one consequence

Power Wall

We can put more transistors on a die than we can afford to switch. The chip's thermal envelope, not its transistor budget, is the binding constraint.

→ Leads to: multicore, dark silicon, DVFS, heterogeneity.

ILP Wall

Diminishing returns from extracting more parallelism out of a sequential instruction stream. Wider issue costs area and power superlinearly and buys less each generation.

→ Leads to: Unit 1, then Unit 2.

Memory Wall

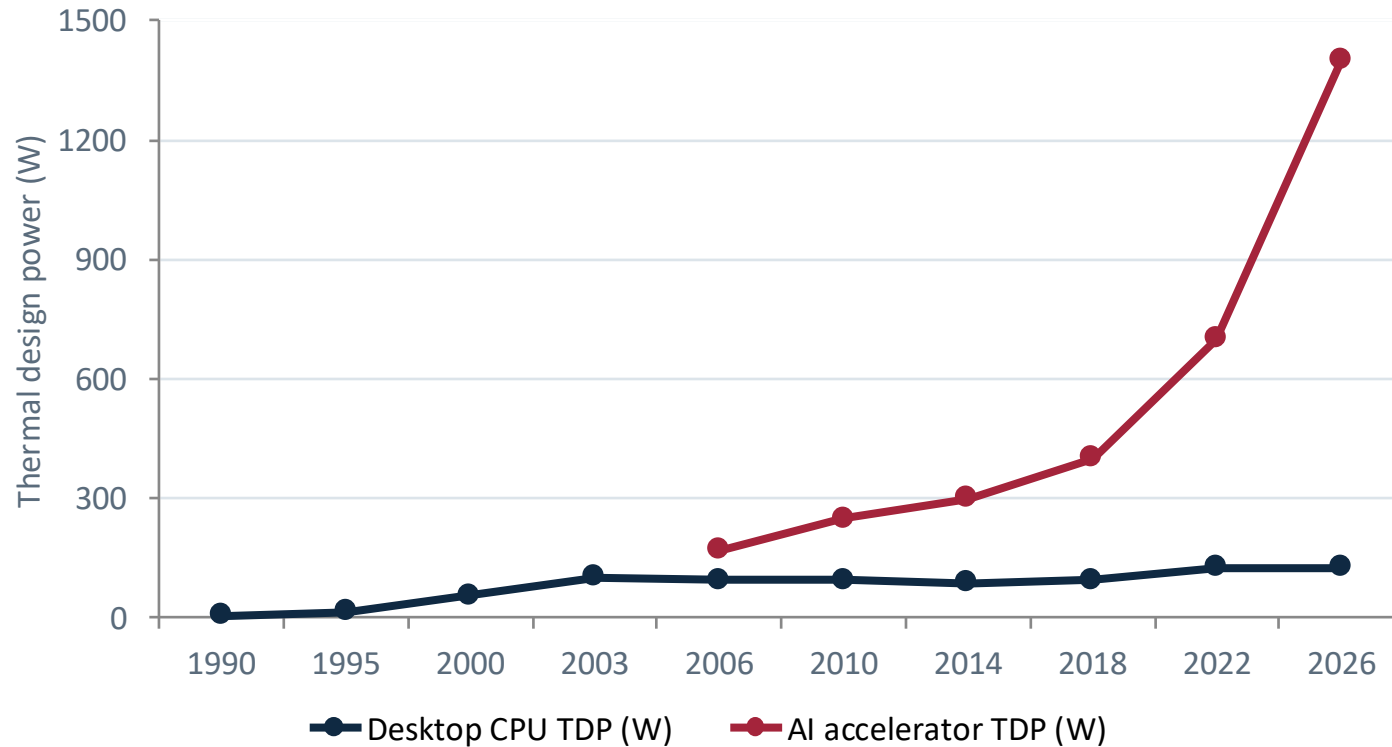
Processor throughput improved far faster than DRAM latency. A last-level cache miss can cost hundreds of cycles — a modern core spends much of its life waiting.

→ Leads to: Unit 3.

Consequence: performance must now be bought explicitly — with parallelism, with locality, or with specialisation.

The power wall

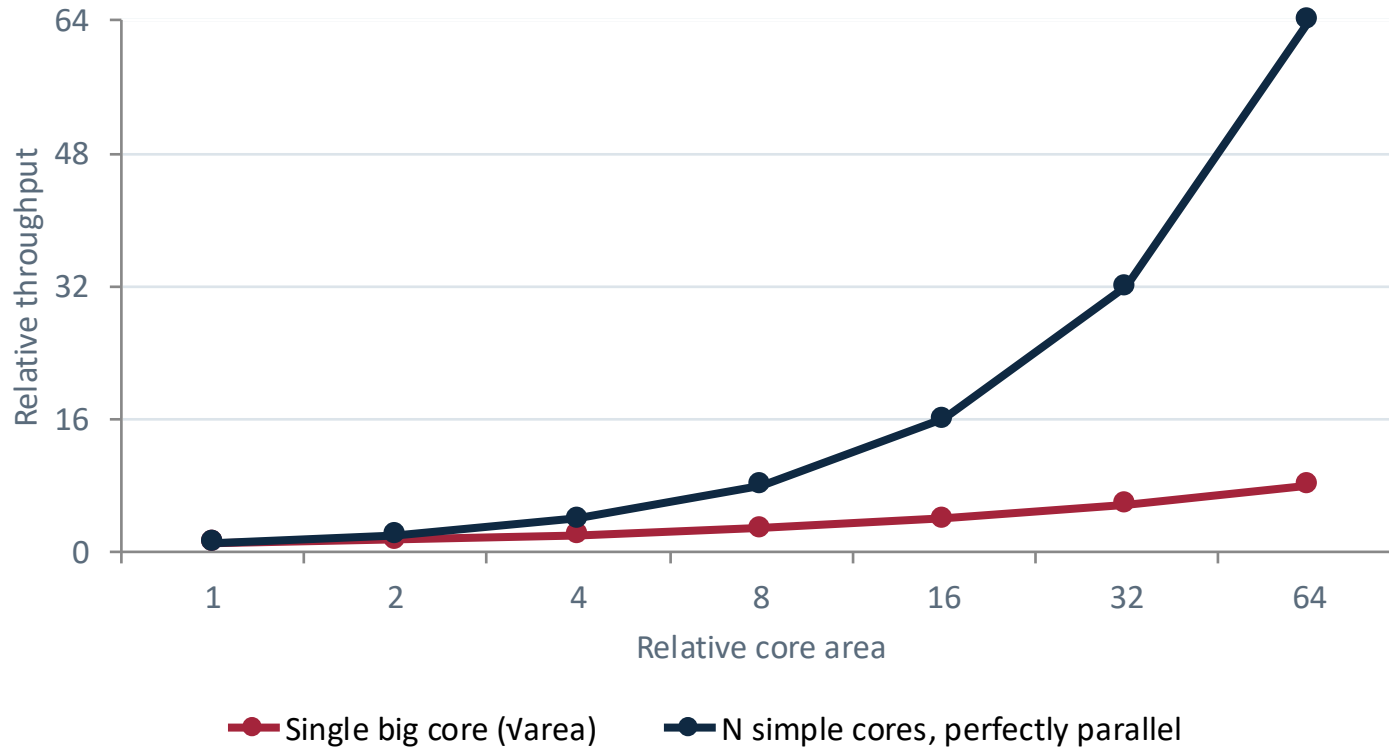
Thermal design power flattened; the transistor budget did not



- Air cooling caps a socket at roughly 100–150 W. That ceiling was reached around 2003 and has barely moved.
- The accelerator curve escapes only by abandoning air: direct liquid cooling, then whole-rack liquid loops at 100+ kW.
- **Cooling is now an architectural parameter, not a packaging afterthought.**
- Corollary: energy per operation (pJ/op), not operations per second, is the number that matters.

The ILP wall

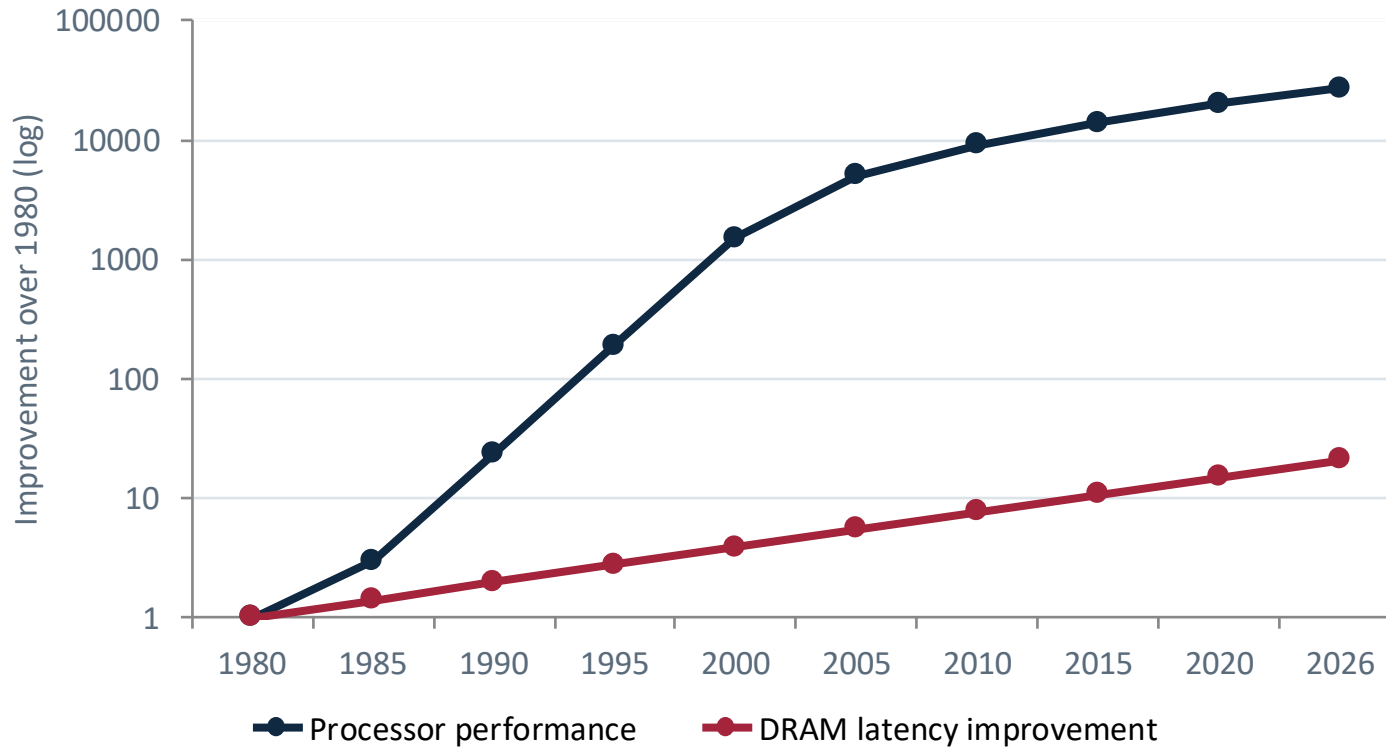
Pollack's Rule: performance rises roughly as the square root of area



- Doubling the area of one core buys about 1.4× its performance — if you can find the instruction-level parallelism to fill it.
- The costs that grow superlinearly: register file ports, bypass network, wakeup/select logic, load-store queue search, branch mispredict penalty.
- Typical integer code sustains an IPC of 1–3 even on an 8-wide machine. The parallelism is simply not in the instruction stream.
- **So: spend the transistors on more cores instead. That is the multicore turn — and Amdahl is waiting.**

The memory wall

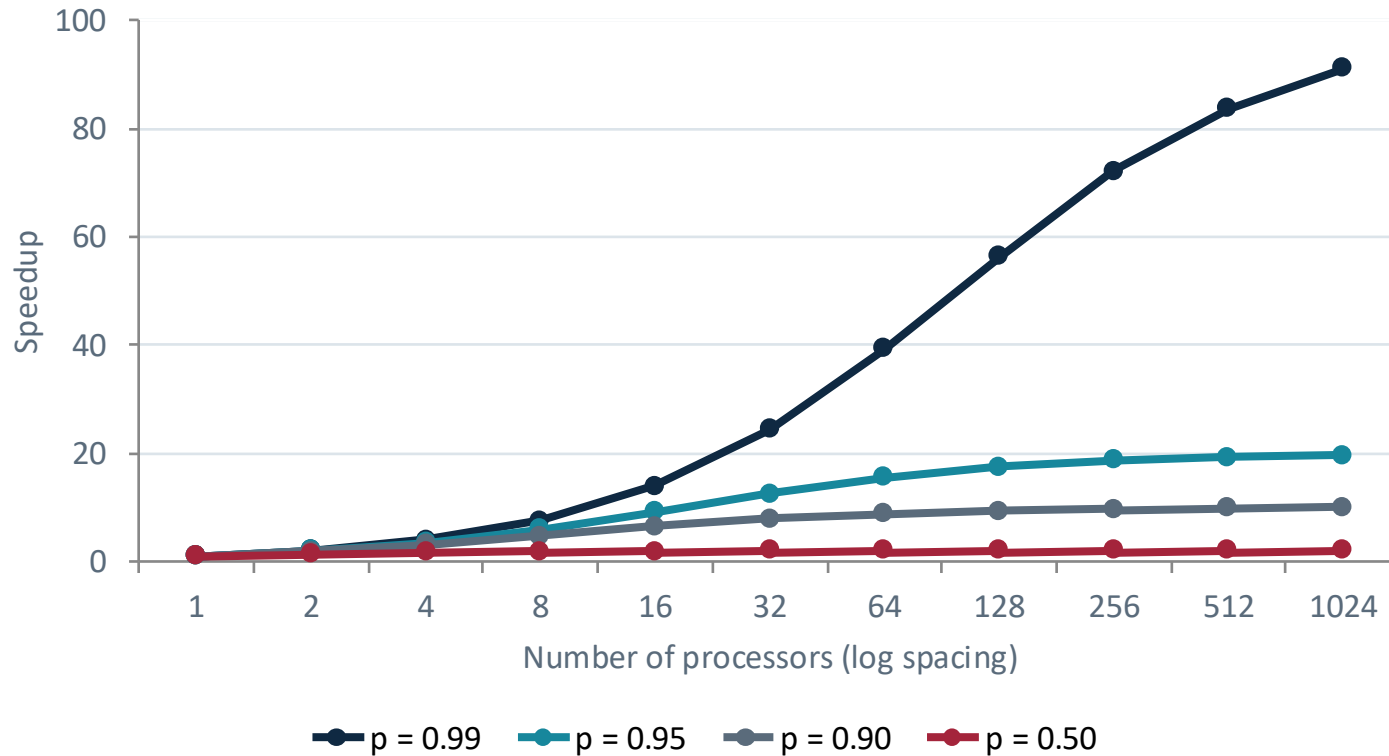
Processor throughput and DRAM latency improved at wildly different rates



- Processors improved $\sim 50\%$ /year for two decades. DRAM latency improved $\sim 7\%$ /year, consistently.
- The gap is now roughly three orders of magnitude. A DRAM access costs 200–400 core cycles.
- Bandwidth we can buy (HBM4 at ~ 20 TB/s). Latency we largely cannot — it is set by physics and DRAM array organisation.
- **Hence: caches, prefetching, multithreading, and moving computation to the data.**
- Unit 3 and the near-memory topic in Unit 4 are both responses to this one graph.

Amdahl's Law

The serial fraction sets a ceiling that no amount of hardware can lift



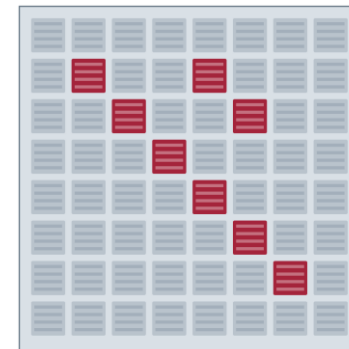
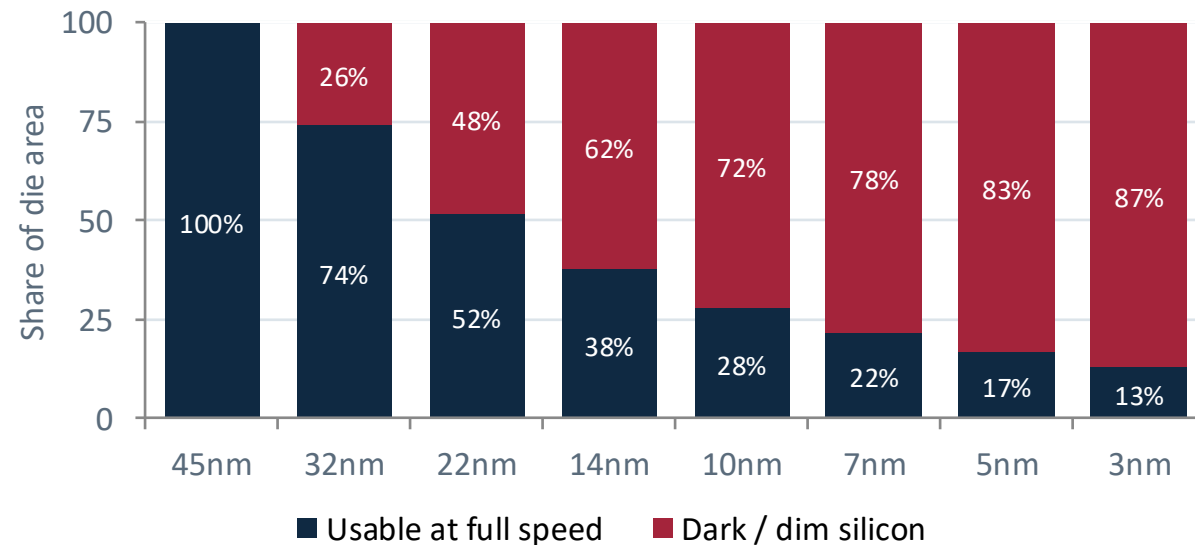
$$\text{Speedup} = 1 / [(1 - p) + p/N]$$

- p = fraction of work that is parallelisable, N = processors.
- With 1% serial work, 1024 cores deliver ~91x — and the ceiling is 100x no matter how many you add.
- With 10% serial work, the ceiling is 10x. You reach half of it at 9 cores.
- **Amdahl's Law is why core counts did not save us — and why specialisation did.**
- Corollary for this course: attack the serial fraction, or make the parallel part cheaper per operation. Usually both.

Dark silicon

When you have more transistors than power, some of the chip must stay off

Post-Dennard arithmetic: transistor count per unit area grows as κ^2 , but the power budget stays fixed. The fraction of the die you can switch simultaneously at full frequency shrinks with every generation.



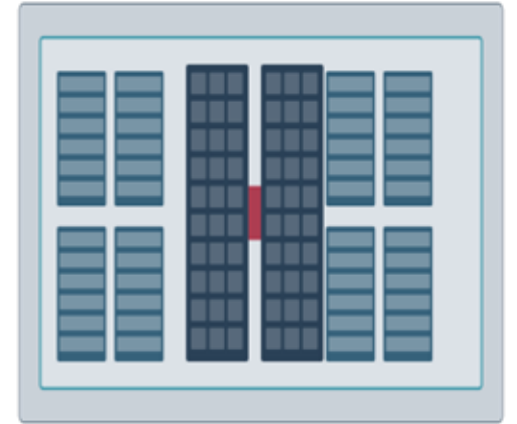
13% lit at 3 nm

- **If most of the die must be dark anyway...**
- ...then transistors are cheap and power is expensive.
- So fill the dark area with specialised units that are idle most of the time but enormously efficient when used.
- **Dark silicon is the economic argument for accelerators.**
- It is why your phone SoC has a dozen fixed-function blocks, and why Unit 4 exists.

5

A New Golden Age

What architects do now that scaling will not do it for them



Where performance comes from now

Four levers, in rough order of remaining headroom

1 Software efficiency

A naive Python triple loop vs. tuned, parallel, vectorised C for matrix multiply: ~60,000× on the same hardware.

Huge, unglamorous

2 Algorithms

Better asymptotics still beat better silicon. FlashAttention changed transformer economics without changing a transistor.

Large, problem-specific

3 Domain-specific architectures

TPUs, tensor cores, genomics accelerators: 10–100× performance/watt on a narrow domain.

Large, narrow

4 General-purpose microarchitecture

Wider issue, better predictors, bigger caches. Real, incremental, expensive.

Small, ~3–5%/yr

Domain-specific architectures

Give up generality, buy back efficiency

What you give up

Programmability across workloads. Branch-heavy irregular code. Backward compatibility. A stable software ecosystem. Long product lifetime as the domain shifts under you.

What you buy

Simpler control (no speculation, no OoO). Data types matched to the problem (INT8, BF16, FP4). Explicit, software-managed memory instead of caches. Massive, regular parallelism. 10–100× better performance per watt.



Systolic array: a TPU's core

Examples we will study

TPU

Systolic array,
software-managed SRAM

GPU tensor cores

SIMT + matrix engines,
Unit 2

AMD AI Engine

Spatial dataflow array,
Unit 4

Genomics accelerators

Sequence alignment,
Unit 4

Near-memory compute

Move compute to data,
Units 3 & 4

The bill for speculation

Spectre and Meltdown: performance optimisations that leaked secrets

1 Speculate

The processor executes past an unresolved branch or a pending permission check, because waiting is expensive.

2 Touch memory

Speculative loads bring lines into the cache — including lines chosen using data the program was never allowed to read.

3 Roll back

The misprediction resolves. Architectural state is restored perfectly. The ISA contract is honoured.

4 Leak

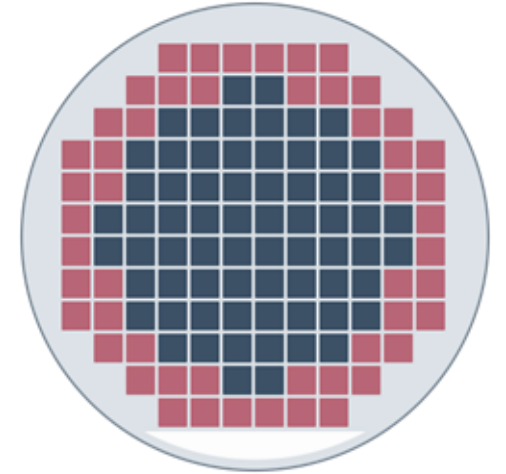
But the cache is not rolled back. Timing the cache recovers the secret. The microarchitecture kept a receipt.

- **Why this belongs in an architecture course**
- The vulnerability is not a bug in any line of code. It is a consequence of a correct implementation of a legal optimisation.
- Mitigations cost real performance — some workloads lost 15–30%.
- **Every microarchitectural state that is shared and observable is a potential channel.**
- Unit 6 revisits this once you have built the speculative machine in Unit 1.

6

The Tools of the Trade

How we will argue about performance for the rest of the semester



The Iron Law of processor performance

Every optimisation in this course moves one of these three terms

$$\text{CPU time} = \text{Instruction Count} \times \text{Cycles per Instruction} \times \text{Clock Cycle Time}$$

Instruction Count

Set by the ISA, the compiler and the algorithm.

Levers: SIMD/vector instructions, fused operations, better code generation, a better algorithm.

Cycles per Instruction

Set by the microarchitecture and the memory system.

Levers: Pipelining, superscalar issue, out-of-order execution, branch prediction, caches, prefetching.

Clock Cycle Time

Set by the circuits, the pipeline depth and the process.

Levers: Deeper pipelines, faster process — both now largely exhausted.

Takeaway When someone claims a speedup, ask which of these three terms moved — and what happened to the other two.

A worked example

Why "faster clock" is not the same as "faster program"

Machine A: IC = 10^9 , CPI = 1.5, clock = 3.0 GHz | Machine B: same binary, CPI = 2.4, clock = 4.5 GHz

- Machine A: $T = 10^9 \times 1.5 \times (1/3.0 \text{ GHz}) = 0.500 \text{ s}$
- Machine B: $T = 10^9 \times 2.4 \times (1/4.5 \text{ GHz}) = 0.533 \text{ s}$
- B has a 50% higher clock and is 6.6% slower. The deeper pipeline that bought the frequency also raised CPI — mispredict penalties and cache-miss stalls now cost more cycles.
- This is not hypothetical: it is roughly the Pentium 4 versus Pentium III story, and it is why the NetBurst line was abandoned.
- **Rule: never compare clock frequencies. Never compare CPI. Compare wall-clock time on a workload you care about — and report the power.**

Takeaway Speedup = $T_{\text{old}} / T_{\text{new}}$. Everything else is a proxy, and proxies lie.

Metrics we will insist on

"Faster" is not a measurement

Latency vs. throughput

Time for one task vs. tasks per unit time. Optimising one routinely damages the other. Say which you mean.

Speedup and efficiency

$S = T_{\text{old}} / T_{\text{new}}$. Efficiency = S / N for N resources. A 16× speedup on 64 cores is a 25% efficiency — report both.

Energy and power

Power (W) is a constraint. Energy (J) is a cost. Energy-delay product (EDP) captures the trade between them.

Means matter

Arithmetic mean for times, harmonic mean for rates, geometric mean for normalised ratios such as SPEC scores. Using the wrong one changes the winner.

Benchmarks

SPEC CPU, SPECjbb, MLPerf, PARSEC. Real applications beat kernels; kernels beat synthetic loops; synthetic loops beat opinions.

Report the baseline

A speedup without a stated baseline, workload, input size and machine configuration is not a result. We will hold papers to this too.

The course, seen as a set of answers

Every unit responds to something we saw today

The ILP wall	Unit 1 — Out-of-order superscalar processors	How much parallelism can we extract from one instruction stream, and what does it cost?
Amdahl + the power wall	Unit 2 — Vector architectures and GPUs	Regular data parallelism at a fraction of the control overhead per operation.
The memory wall	Unit 3 — Caches, DSM, consistency, NVM	Locality, bandwidth, and the semantics of sharing memory between many cores.
Dark silicon	Unit 4 — Domain-specific accelerators	Spend cheap transistors to save expensive joules.
Design-space explosion	Unit 5 — AI/ML techniques in architecture	When the policy space is too large to hand-tune, learn the policy.
The cost of speculation	Unit 6 — Side channels: Spectre, Meltdown	What the microarchitecture remembers that the ISA promised to forget.

Before the next lecture

Read

Hennessy & Patterson, 6th ed., Chapter 1 — sections 1.1 to 1.9.

Moore's original 1965 Electronics article (three pages; read it, it is not what you expect).

Optional but recommended

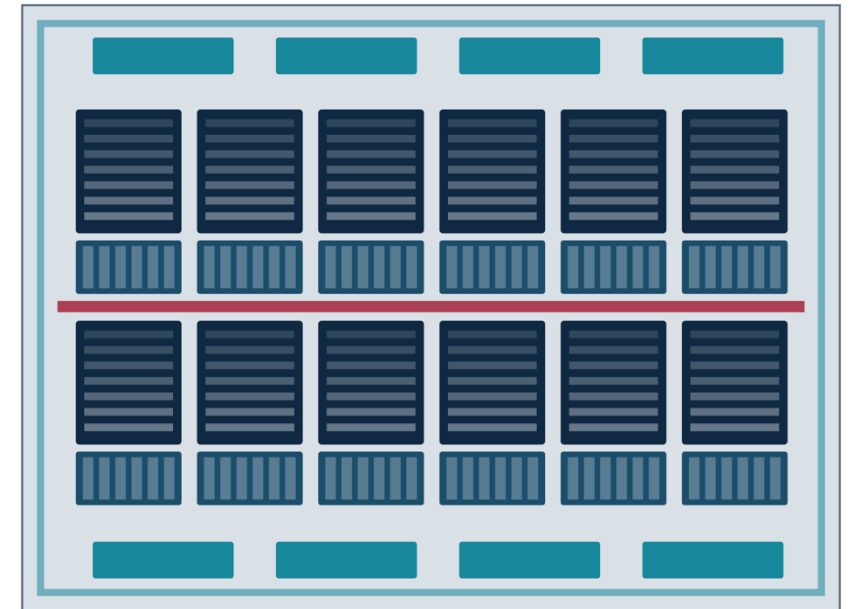
Hennessy & Patterson, "A New Golden Age for Computer Architecture", CACM, February 2019 — their Turing Award lecture in article form.

Come to the tutorial ready to answer

- Machine A runs a benchmark in 8.2 s at 95 W. Machine B runs it in 6.1 s at 165 W. Which is better? Defend your answer with a metric.
- If a workload is 92% parallelisable, what is the maximum speedup? At how many cores do you reach 80% of that maximum?
- Name one optimisation from your own code that reduced instruction count, and one that reduced CPI. How do you know which was which?

The one sentence to remember

**For fifty years, physics
made computers faster.
That has stopped.
From here, architecture
has to.**



Many-core die floorplan, schematic